ITERATOR PATTERN

Behavioural Design Pattern

Iterator Pattern

The Challenge

How do you go through items in a collection (list, array, tree, etc.) without needing to know how the collection stores those items internally?

The Solution

Iterator Pattern gives you a standard way to loop through any collection while keeping its internal organization hidden. Every collection provides the same interface for accessing items one by one.

Main Components

- **Iterator:** Defines the methods to move through items in a collection (like has_next() and next()).
- **Collection:** Stores the items and creates an iterator that knows how to access them properly.

Iterator Pattern Benefits

Clean Separation: The iteration logic stays separate from the collection, so you can change one without affecting the other.

Consistent Interface: The same Iterator interface works with all collection types, making your code more flexible.

Simpler Client Code: Client doesn't need to know the internal data structure, reducing dependencies and making code easier to maintain.

Flexible Iteration: You can create different iterator types (forward, backward, filtered) without changing the collection itself.